



BRNO UNIVERSITY OF TECHNOLOGY

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF COMPUTER GRAPHICS AND MULTIMEDIA

ÚSTAV POČÍTAČOVÉ GRAFIKY A MULTIMÉDIÍ

KNN CHECKPOINT

https://github.com/Martin-Toma/KNN_project

AUTHORS

**Bc. Rostislav Červenka
Bc. Attila Kovács
Bc. Martin Tomašovič**

BRNO 2025

Contents

1	Checkpoint	2
1.1	Task definition	2
1.2	Existing solutions	2
1.3	Dataset	3
1.4	Baseline	5
1.5	Baseline Testing	5
1.5.1	Score Tests	5
1.5.2	Grammar Test	6
1.5.3	Review Similarity Test	6
1.5.4	Sentiment Alignment Test	6
1.5.5	Genre Test	6
1.6	Baseline Results	7
1.7	Solution plan	9
1.7.1	Model	9
1.7.2	Experiments	9
1.8	Dataset preparation	10
1.9	Model selection	10
1.10	Finetuning	10
1.10.1	Setup	11
1.10.2	LoRA	11
1.10.3	QLoRA	13
1.10.4	7B model	13
1.10.5	Multi-head model	14
1.11	Inference	15
1.12	Results	16
1.13	Conclusion	19
1.14	Links	21

Chapter 1

Checkpoint

1.1 Task definition

This project aims to fine-tune a large language model to generate preliminary reviews and estimate the movie genre based only on the subtitles of a movie. This could reduce the number of badly perceived movies released each year caused by a bad story. We accomplish this by giving script reviewers a tool to reduce their workloads. Because movie scripts actually are not publicly available, we have decided to substitute them with subtitles, which, in our opinion, is the closest publicly released material to them. This decision has its pros and cons:

- Subtitles do not contain as much information as scripts. For example, a scriptwriter can describe how a scene should look, while this information is completely absent from the subtitle.
- Working only with the subtitles means that the language model could also be used by people not in the movie industry to judge if a given movie is worth a watch.
- Subtitles are usually shorter than scripts.

The reason we didn't go with a shorter movie plots is because when testing various instruct variants of the models with the plot as the input even the smallest model was able to properly guess the name of the movie, possibly because it has already seen the plot. Part of this comparison can be found in `out.txt`. With the subtitle as the input, the model would return the following as a JSON object:

- Rating prediction ranging from 1 to 10
- Genre or genres the movie belongs to.
- A review of the movie highlighting its strengths and weaknesses and possible corrections, which would make the movie better received.

1.2 Existing solutions

When searching for existing solutions, we did not find a model that performs this task directly. The idea came from reading an article about the [Largo.ai](#) company. The company offers AI-based content analysis, but unfortunately does not offer open-source solutions.

We found different solutions for related tasks. There were attempts to train GPT2 on reviews and then generate new reviews¹, the problem is that it only saw reviews, not raw movie data. DeepAI offers a model called [AI Movie Critic](#) generating „honest“, spoiler-free movie reviews, but these are based on the movie title, which means that it is again based on existing reviews of a movie. Another similar tool is based on a fine-tuned GPT tool called [Callaia](#) made by Cinelytic. It provides a synopsis, a list of comparable films, grades for areas like dialogue and originality, and actor recommendations based on a movie script.

We want to train a model that learns how to create the review, not just copy it. As our task involves multiple targets, we will fine-tune a model for multitasking with LoRA, using knowledge from this [paper](#) which we have described in the video.

1.3 Dataset

During the design phase, we have decided that we need a dataset which contains the following:

- Subtitles
- Reviews
- Genres
- Average ratings

The first obstacle was finding a dataset containing all the properties mentioned above. Unfortunately, there is no single dataset, so we either had to combine multiple datasets or create/scrape our own dataset from scratch. We chose to combine multiple datasets, as obtaining data from the web would have taken too long, and we did not have the required time and resources.

Our dataset is the combination of:

- Unofficial opensubtitles.org [dataset](#) – The reason we chose this dataset as the base is because it was the only one that contained the most subtitles and has extensive metadata associated with each subtitle. This makes it easier to combine it with other datasets.
- Reviews from [Amazon](#) – This was the only dataset that contained a wide variety of reviews while also including the movie title in the metadata for each review.
- Ratings and genres from the [IMDB non-commercial dataset](#) – During the evaluation of the dataset above, we have found that the average ratings in the reviews dataset have a really small distribution, the majority of the movies had an average rating over 4 on a 1-5 scale. For this reason only, we have decided to pursue alternative datasets for the ratings, and that’s when we stumbled upon the IMDB datasets. Unfortunately, they don’t contain reviews, but they do contain the average ratings with a much wider distribution. Besides that, IMDB also publishes the genres of each movie, so we have decided to use that part of the dataset as well.

¹<https://medium.com/data-science/how-to-write-movie-reviews-with-ai-d17f758f2ed5>

None of these datasets can be considered particularly clean. During the combination process, we've tried to clean them up as much as possible programmatically, but there's still a chance that we've missed some errors. This is mainly due to the size of the dataset and time constraints. The main problems we've encountered were:

- Duplicate movies in the subtitles dataset – This is caused by a movie having multiple releases, and each release can have different timings, some added/removed scenes, . . . This is not a problem for us, as we are only interested in the subtitles and not the actual movie. The duplicates were removed by keeping only the first found subtitle for each movie.
- Invalid subtitles (subtitles which are not for the entire movie, but only for a part of it, for example, for the trailer) – This is caused because the dataset consists of user-uploaded subtitles, and there's a chance that some of these subtitles are incomplete.
- Invalid reviews – reviews for a series are assigned to a movie with the same name. This is because the reviews dataset contains reviews for movies and series, and the only way to connect them is based on the movie's title. If a movie and a serial share a title, the reviews for the serial will also appear in the final dataset.

The dataset² is divided into 1,000 samples for testing, 1,000 for evaluation, and approximately 31,000 for training.

²<https://akirabox.com/9QWmpZ7XzEB6/file>

1.4 Baseline

For the baseline, we chose to use the high quality model DeepSeek-V3 Base 671B parameter open Mixture-of-Experts with 37B active parameters per forward pass. We access the model through the [OpenRouter API](#) using the OpenAI library.

The model has context window of 131K tokens. However, some movies in the dataset exceed the limit size when constructing prompts for them. To mitigate this problem we trimmed part of the subtitles in the middle, between 25% and 80%, just to fit the context size. There was only one movie in the test dataset which required the trimming. To reduce randomness in the generated outputs, we set the temperature to a low value. We crafted prompt to restrict the model output to our specific dataset constraints. We used this prompt to generate data for testing.

```
'Please provide a review in the following format: {"rating": <rating>, "genres":  
<genres>, "review": <review_text>}. Make sure the rating is a number in the range  
of 1.0 to 10.0. In the genres field, include 1-3 values from [Action, ...,  
Western]. The review should evaluate the movie in a few sentences. Here are the  
movie subtitles: '
```

For the testing, we extracted the required JSON part from the model response using regex. Not only did we test the output using prepared test cases, but we also computed the perplexity for each prompt. The perplexity statistics are presented in Table 1.1, indicating a relatively low variance in the predictions. The low perplexity values are caused by setting the temperature low. The minimum and average perplexity values are quite low, suggesting the model's high confidence in its outputs. The maximum value remains reasonable. Although low perplexity does not guarantee correctness, it indicates that the model is internally confident in its generated sequences.

Statistic	Value
Minimum	1.0800
Maximum	1.7959
Average	1.1944
Standard Deviation	0.0748

Table 1.1: Perplexity Statistics

1.5 Baseline Testing

1.5.1 Score Tests

For evaluating the model's ability to predict movie scores, three basic metrics were used:

Mean Absolute Error (MAE): Measures the average absolute difference between the predicted and actual scores.

Root Mean Squared Error (RMSE): Measures the squared absolute difference, meaning that it punishes larger errors more heavily.

% Within Tolerance: Measures the proportion of predictions falling within ± 1 points of the true score.

1.5.2 Grammar Test

Checks the grammatical accuracy of generated reviews using [LanguageTool](#), an open-source grammar and style checker that detects grammatical errors, punctuation errors, and stylistic issues.

The metric: The average number of errors per review.

1.5.3 Review Similarity Test

Measures how semantically similar the generated reviews are to the real human-written reviews using [SBERT](#) (Sentence-BERT).

We use the `all-MiniLM-L6-v2` model from `SentenceTransformers` to convert both generated and original reviews into high-dimensional vectors. After encoding the texts, we compute the cosine similarity between the generated review and each original human-written one. Higher similarity scores indicate that the generated review conveys similar ideas and meanings to the original reviews. Human-written reviews with fewer than 50 characters are automatically skipped and not tested.

The metric: The average similarity to a human review (0 to 1).

To evaluate the effectiveness and behavior of SBERT in this context, we also performed two additional tests using only human-written reviews:

Original-to-original similarity: This test measures how semantically similar human-written reviews are to one another within the same movie.

Cross-movie similarity: This variant selects one random human-written review from a given movie and compares it to every human-written review in the following movie.

1.5.4 Sentiment Alignment Test

Verifies whether the sentiment of a generated review matches its given rating.

To analyze the review text and assign a sentiment score, we use [VADER](#) (Valence Aware Dictionary and Sentiment Reasoner), a rule-based sentiment analysis tool that outputs a compound score between -1 (negative) and +1 (positive).

Movie scores are scaled to $[-1, 1]$ and compared to a VADER score. If the difference between expected and predicted sentiment exceeds 0.5, it is flagged as a mismatch.

The metric: The percentage of reviews with mismatched sentiment.

1.5.5 Genre Test

To evaluate the model’s ability to correctly predict movie genres by comparing generated genres with original human-assigned genres, we use standard classification metrics:

Precision: Measures the proportion of correctly predicted genres out of all predicted genres.

Recall: Measures the proportion of correctly predicted genres out of all actual genres.

F1-score: The harmonic mean of precision and recall, balancing the two metrics.

These scores help assess how well the model generates relevant genres without adding incorrect ones.

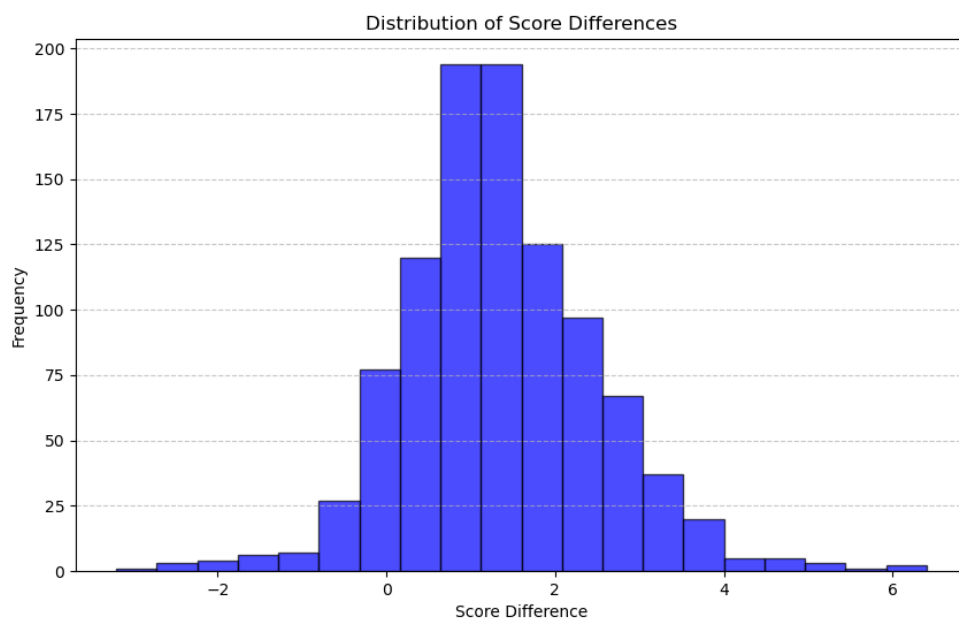
Metric	Value
Rating Accuracy – MAE	1.47
Rating Accuracy – RMSE	1.77
Rating Accuracy – % within ± 1 rating	37.79%
Avg. similarity of human and generated reviews	0.42
Avg. similarity of only human reviews	0.42
Avg. similarity of cross-movie human reviews	0.26
Mismatched sentiment-rating pairs	46.93%
Avg. grammar errors per review	0.31
Genre Accuracy – Precision	0.58
Genre Accuracy – Recall	0.78
Genre Accuracy – F1 Score	0.66

Table 1.2: Evaluation Metrics for Generated Reviews

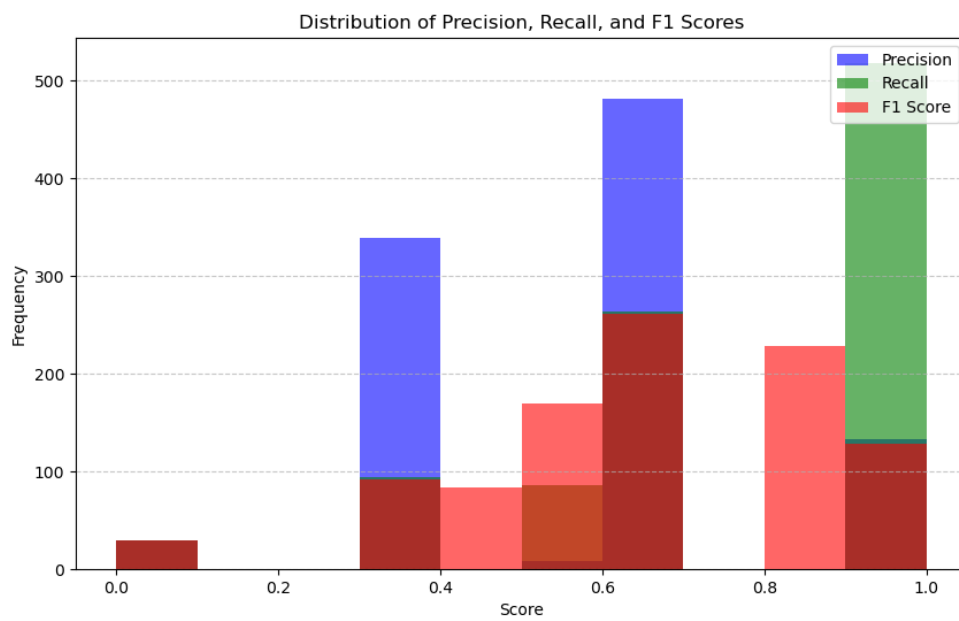
1.6 Baseline Results

The average similarity between generated and human-written reviews was 0.42—the same as the average similarity among human reviews within the same movie. This indicates that the SOTA model produces reviews that are semantically on par with real ones. In contrast, cross-movie human review similarity was lower at 0.26, confirming SBERT’s ability to capture meaningful textual differences across distinct contexts.

Some additional interesting facts learned from graphs can be seen [here](#).



(a) The model assigned excessively high scores.



(b) Recall is higher than precision due to the model frequently predicting more genres than necessary.

1.7 Solution plan

In our experiments, we will fine-tune a much smaller model on our dataset. For that purpose, we will use parameter-efficient fine-tuning to reduce computational complexity. We will experiment with the training settings. In the end, we will compare the results with our baseline.

1.7.1 Model

We wanted a smaller language model, to be able to fine-tune it with the available resources. We initially chose the [MPT 7B](#) model. But due to various compatibility issues we had to abandon it and look for alternatives. Our next choices were Mistral 7B and the relatively new Gemma3 4B. Because Gemma generated a more human like response we've decided to mainly focus on that. We used a structured prompt to ensure that the output is consistent. This prompt will be similar to the baseline prompt. We've decided to use the chat template of the instruct models, to make it possible to compare them.

We will train two types of models, text-only output and a model with separate heads for each output. 1. The text-only model will output structured text in JSON format like the one we used in the baseline evaluation. 2. Model with separate heads will have a text generation head for reviews, a regression head for the rating scores, and a classification head for genres. The regression part will be a small fully connected layer in the output for numerical score prediction. The classification part will be a multi-label classification layer.

We will use a loss function with three components. For generating the review text, we will use cross-entropy loss. For regression, we will use the Huber loss with $\delta \approx 1.0$. For genre classification, we will use Binary Cross-Entropy with sigmoid because each genre is an independent classification problem and a movie can belong to multiple genres simultaneously. The loss functions will be weighted and summed together: $\mathcal{L}_{\text{total}} = \alpha\mathcal{L}_{\text{text}} + \beta\mathcal{L}_{\text{regression}} + \gamma\mathcal{L}_{\text{genre}}$.

1.7.2 Experiments

Comparison of LoRA and QLoRA fine-tuning methods. Each fine-tuning will also incorporate finding the best settings for hyper-parameters like: LoRA rank, alpha, dropout, and others. We will also need to find hyper-parameters for the aggregated loss function for the model with an adjusted architecture. The expected outcome is to determine which method produces the best balance of performance and computational efficiency.

1.8 Dataset preparation

For the fine-tuning we needed to further transform our dataset. The prepared dataset was structured so that each movie included a subtitle, a list of genres, a score, and multiple reviews.

First, we transformed the dataset to the instruction format³. We changed the format so 'prompt' part contained only the subtitle and the 'completion' part contained score, genres list and a single review. This was saved as a list of dictionaries in json format.

After that, a problem with too much data appeared we already had 33k movies in the dataset and each movie had multiple reviews. To constrain the size we randomly selected at most five reviews per movie, later to save on computational resources we decided to use three reviews per movie for training.

The prompt part still contained the complete subtitle, which for some movies didn't fit the target context size. We created a script which calculated the size of the prompt and the completion, and trimmed just the subtitle part to fit to the desired size. The trimming erased the middle part of subtitles, keeping symmetrically large start and end, which we assume are the main parts which influence a viewer's opinion. We trimmed the dataset so that the input and the output would fit into 31k tokens.

For some reason SFTTrainer was acting weirdly and we had to manually apply the chat template. To speed up the training especially in a CPU constrained environment we've also saved the dataset in a tokenized format.

1.9 Model selection

We ran most of the fine-tuning on Metacentrum. There appeared an issue with installation and compilation of the triton-pre-mlir library required by the MPT 7B models. We've also tried to make it work locally where we had sudo rights, but were unable to make it generate something sensible, or it failed because the modern Transformers library deprecated some of the methods it uses. After extensive effort to fix the issues on Metacentrum we were unable to set it up due to not having sudo privileges required to install necessary tools to compile the required version. Even after trying to install it by any means or to avoid the installation and using different attention implementation, the triton-pre-mlir is still required, so we are forced to use a different model. Because we had issues with deprecated functions we've decided to use one of the newest available models gemma 3 4B. This model offers the same context and considering it's size has a remarkable quality in creative tasks. We also fine-tuned a 7B model specifically Mistral 7B Instruct v0.3 to compare models with different sizes.

1.10 Finetuning

We primarily fine-tuned models with LoRA and performed most of the experiments with it. We also created a multi-head model as described earlier and fine-tuned a model with QLoRA.

³https://huggingface.co/docs/trl/sft_trainer#dataset-format-support

1.10.1 Setup

All of the experiments were tested and verified to run correctly on Metacentrum.

1.10.2 LoRA

We fine-tuned Gemma 3 4B model. Even with LoRA we used quantization, but only 8-bit, this also saved on computational resources and also sped up the process. We used a learning rate of $5e-4$ and set the weight decay to 0.01. As we had a large input, we had to use a batch size of 1 which lead to instabilities in training, meaning that one update caused gradient explosion or vanishing. We overcame this by setting the gradient accumulation steps to 8. When the gradient accumulated over multiple steps, it stabilized the training. At the end of the training we saved the best model according to the loss metric. All models training loss and accuracy are shown in figure 1.2.

To compare the dropout influence, we tested models with all the same training parameters except the dropout. From observation of loss and accuracy in figure 1.3, it seems that the lower the dropout the better.

We perform measurements of different ranks. In figure 1.4 the comparison of the influence of rank on training loss and accuracy is displayed, where ranks 32 and 64 are the best performing settings.

We tested different alpha settings of a model with fixed rank. We tried to set the alpha to double the rank, half the rank and the same as the rank 1.5. Better loss and accuracy seems to be achieved with lower alpha settings.

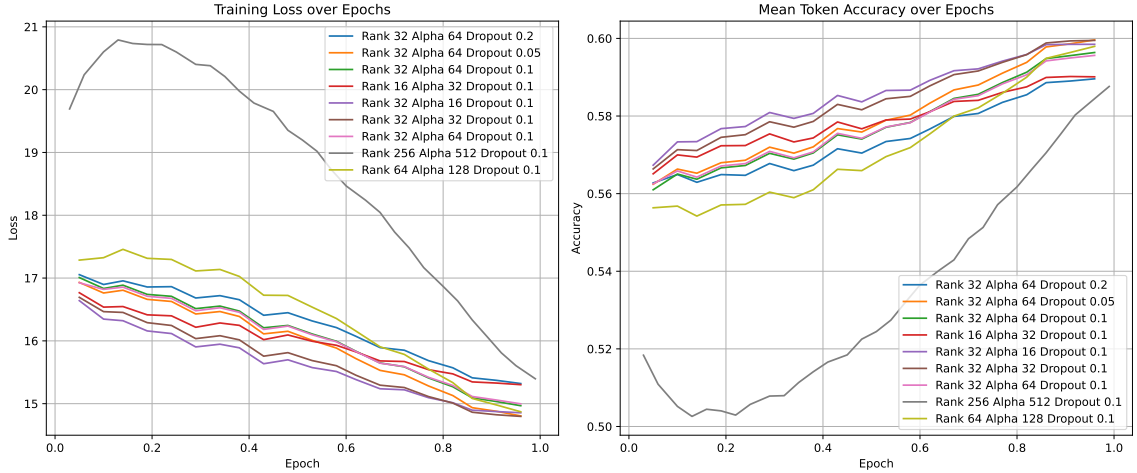


Figure 1.2: Comparison of different model settings loss and accuracy during fine-tuning.

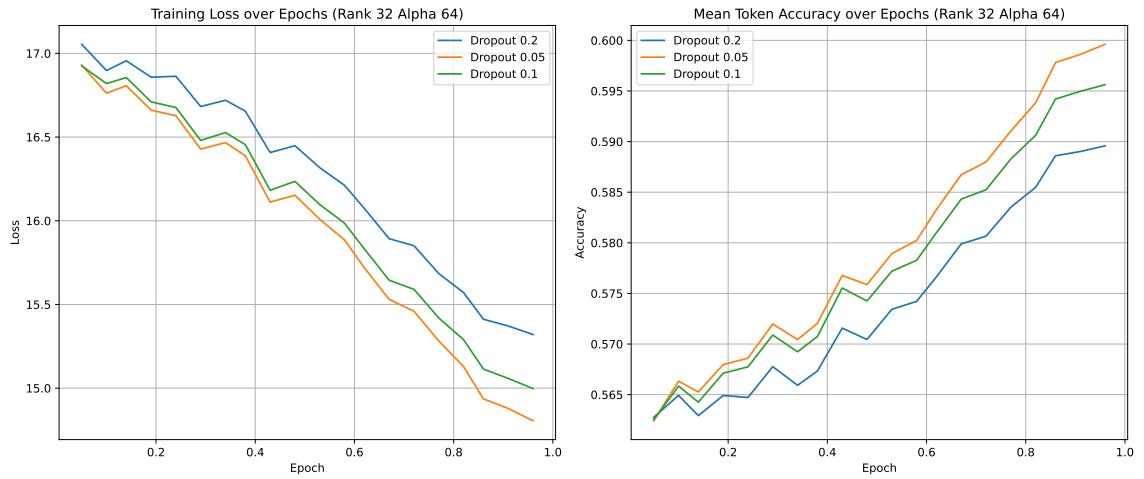


Figure 1.3: Comparison of dropout model settings loss and accuracy during fine-tuning.

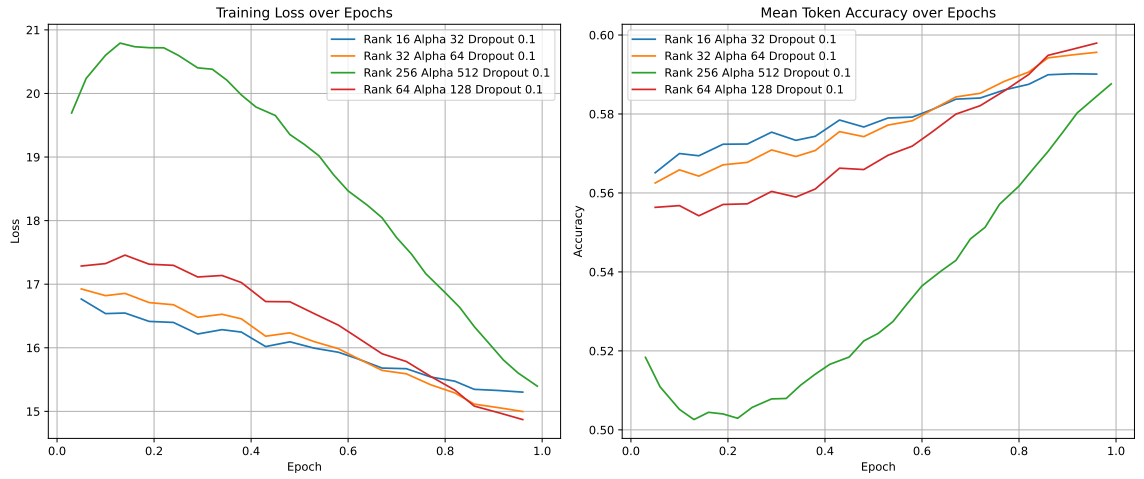


Figure 1.4: Comparison of different model rank settings.

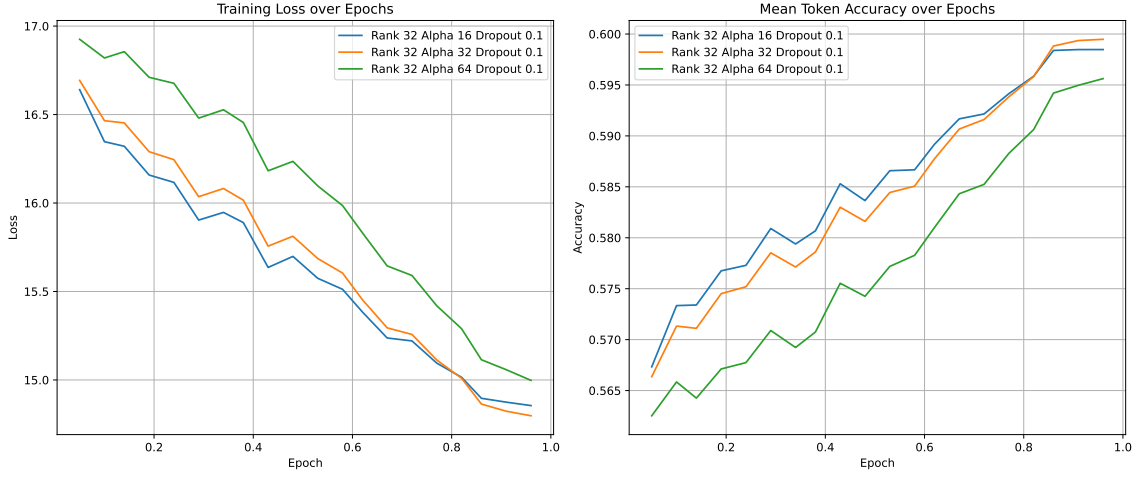


Figure 1.5: Comparison of alpha settings loss and accuracy during fine-tuning.

1.10.3 QLoRA

We also fine-tuned one model with 4-bit quantization alias QLoRA. As we can see, during training there was not much difference between the two 8-bit and 4-bit quantized models. This can be seen in figure 1.6.

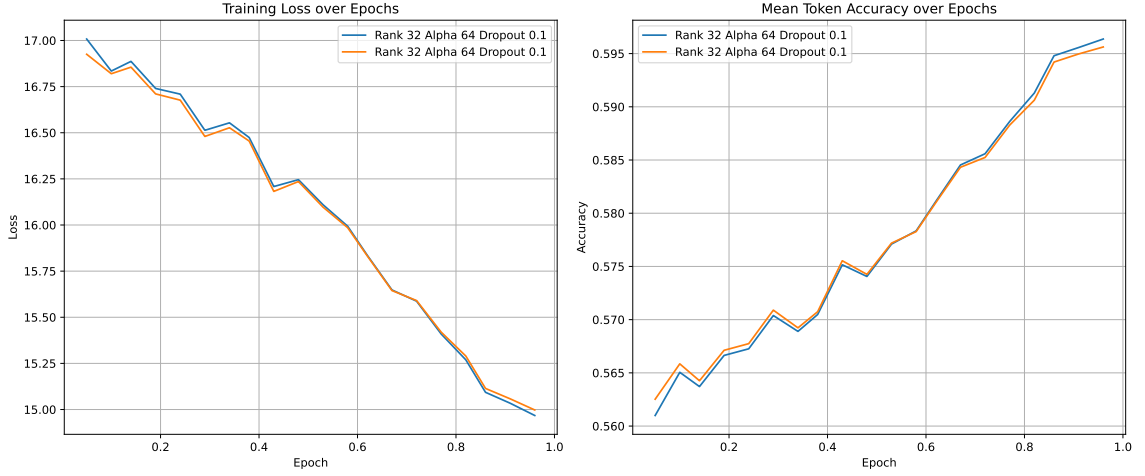


Figure 1.6: Comparison of QLoRA and LoRA model settings loss and accuracy during fine-tuning.

1.10.4 7B model

We also performed fine-tuning on a 7B model with a different architecture to see if it makes any difference. The model was specifically Mistral 7B Instruct v0.3. It was more difficult to find settings for this model as in most experiments we performed, we observed gradient explosion or vanishing gradients. The first nearly successful training was with rank = 32 and alpha = 64, when the gradients exploded after 0.33 epoch and with rank = 16, alpha = 32 when the gradients exploded around 0.58 epoch. It shows that the LoRA rank acts as a form of implicit regularization.

After adjusting the max grad norm, changing the optimizer, and lowering the learning rate, we achieved more stable training. The working setting is using QLoRA with rank = 16 and alpha = 32, dropout set to 0.1, set weight decay to 0.01 and learning rate 1e-6 . We also had to add an adafactor optimizer. We set a lower max grad norm to 0.3 or 0.1 as these two settings have shown almost the same training loss and accuracy progress during training, we further tested the variant with 0.1. The training progress is shown in figure 1.7

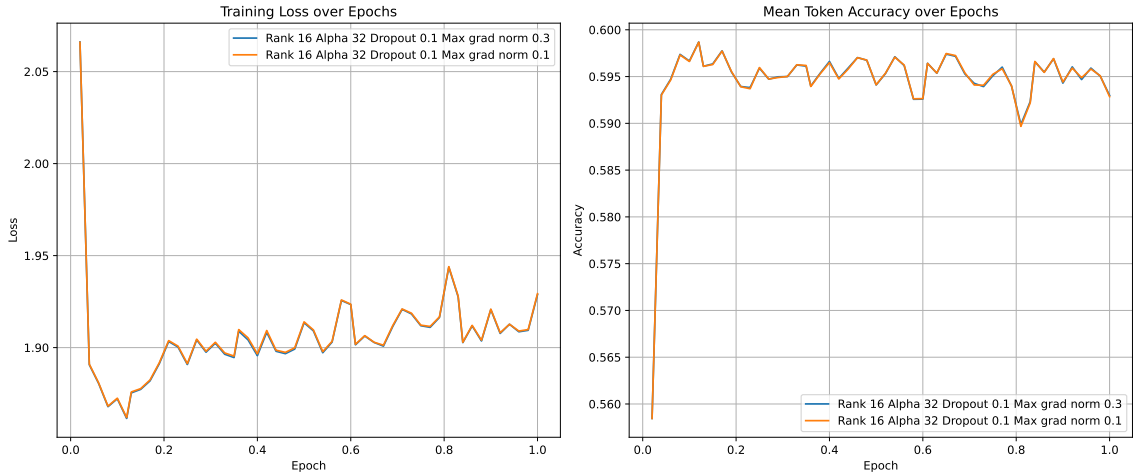


Figure 1.7: Comparison of Mistral 7B settings of max grad norm, showing loss and accuracy during fine-tuning.

1.10.5 Multi-head model

We also experimented with adding more heads to Gemma 3. This was accomplished by connecting 2 linear layers to the last hidden layer. We utilized LoRA for the training with a rank of 32 and an alpha of 64, The loss function is the one mentioned above, with the following weights: $\alpha = 0.1, \beta = 10, \gamma = 10$. These weights were chosen to prioritize the learning of the new heads. Exposing the hidden layers ballooned the VRAM usage so much that we had to decrease the maximum length of our inputs to 8K and limit the fine tuning to only parts of the model. Using these tricks the VRAM requirements were still the highest(>45GB), considering that the base model with the same parameters used about 12GB, this seemed really inefficient. The high VRAM requirement complicated the training procedure a bit, because only 10 nodes have hardware with enough VRAM to train this model on Metacentrum, also these nodes were the most sought upon. We were only able to acquire computation capacity on them during the last few days before the deadline. The training time was also relatively high for 1 epoch it was comparable to finetuning the base model with 128K long inputs. Due to time constraints we've decided to only use the first 10000 inputs from the training dataset. But considering that our first experimental fine-tuning was also constrained this way and had okay results, we thought that it would be sufficient. Unfortunately this wasn't the case and the custom heads perform badly, although the base head trained this way had one of the best results. We think that this is caused by limiting the components that we've fine-tuned. We think the reason that the custom heads generate garbled data is because they are under-trained and with a longer train time

it could be improved. Although except for a fall at the beginning of the training the loss function was oscillating, with a slight decline, this can be seen on figure 1.8.

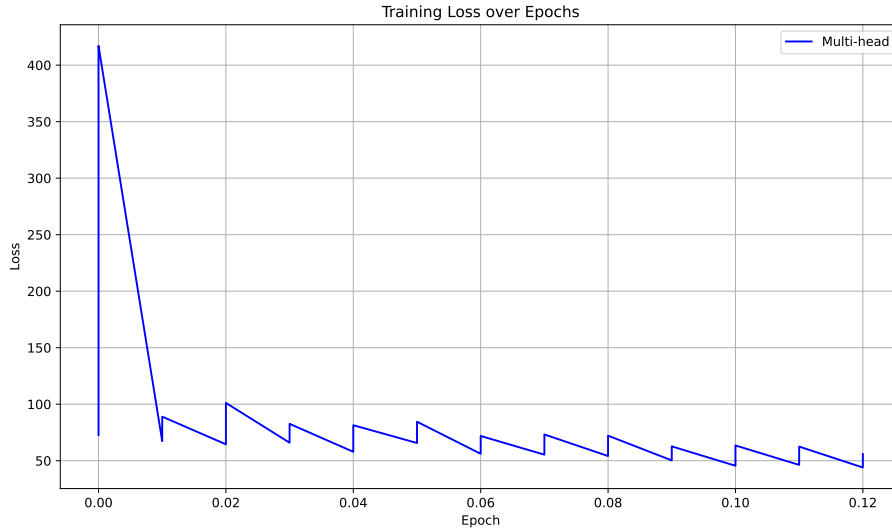


Figure 1.8: The evaluation of the loss function during the training of the multi head model.

We think that this is because one or more of the following reasons:

- Gemma 3 doesn't support adding new heads. Considering that it's a multi-modal model, with image processing capabilities, and according to personal overview also generates and understands foreign languages really well, the architecture to accomplish this might make new heads perform worse.
- Errors in the code. The way we've implemented the custom heads is a bit bare-bones. It's possible that we are doing something incorrectly. We initially wanted to use a library like [this](#). But none of the libraries were compatible with this model.

1.11 Inference

To increase the speed of the inference, we first pre-tokenized the test dataset, so that we could use it in multiple models. As we were running the inference on MetaCentrum, we split the dataset into 10 blocks, each with 100 exams, then ran 10 jobs at the same time, significantly shortening the time the inference took.

Although the testing part of the dataset contained 1000 movies, during the inference some problems showed up and we were not able to get all 1000 responses. The problems consisted of CUDA errors, torch errors and the main problem being the models did not always respect the JSON structure, that they were asked to follow. This resulted in us receiving around 750 responses for each model with the correct format, that could be further tested. The most problematic model turned out to be Mistral, which did not respect the correct JSON formatting at all, often forgetting one of the required parts, be it the genres, rating or the review, it also often forgot to put brackets around its JSON, this resulted in us getting only around 250 proper responses from it.

During the inference, we also calculated the perplexity of the models responses.

1.12 Results

We tested a lot of models and their settings the full list with their respective test results can be seen in [this](#) table. Now we are going to focus on the most interesting results. What the testing metrics represent and how they are calculated can be found [here](#).

Overview of Models

We evaluate:

- **Deepseek:** Our custom retrieval-augmented baseline.
- **Base Gemma instruct:** 4 billion-parameter instruct model gemma-3-4b-it, which was not fine-tuned for our purposes. We could not use the non-instruct version which we were using during finetuning, as it was unable to answer given prompts.
- **Gemma variants:** Gemma-3-4b-pt model differing in architecture scale (e.g. “a8 r16” means 8-layer, 16-head), training status (unfinished vs. finished), dropout settings, and QLoRA quantization.
- **Multi-Head:** Multi-head attention model, we only use the main head to generate reviews, genres predictions and rating, the other heads don’t work properly.
- **Mistral 7B:** Our 7 billion-parameter model.

Rating Prediction Accuracy

- *MAE and RMSE:* The Multi-Head model achieves the lowest MAE (0.96) and RMSE (1.25), indicating the highest rating-prediction fidelity. By contrast, Mistral 7B shows the poorest rating accuracy (MAE 1.90, RMSE 2.34).
- *% within ± 1 :* Multi-Head again leads at 64.40%, while Mistral lags at 29.73%.
- Deepseek’s MAE of 1.47 and 37.79% within ± 1 are comparable to the mid-range Gemma variants (e.g. Gemma a32r16 finished with 1.34 MAE and 50.92% within ± 1). All of the models vastly outperformed base Gemma.

Review generation

- *Similarity to Human Reviews:* Across the fine-tuned Gemma variants and Multi-Head model, semantic similarity scores cluster between 0.347 and 0.363, with the Multi-Head reaching 0.361 and the best fine-tuned variant being Gemma a8 r32 at 0.372—indicating that most models generate text of comparable human-like style, compared to the base Gemma model which scored 0.284. Even though none of them can compare with Deepseek’s 0.42.
- *Sentiment–Rating Mismatch:* Most fine-tuned models achieve mismatch rates in the 33–43% range, with the strongest alignment seen in Mistral at 28.96% and the weakest among the Gemma variants around 43.37%, showing generally consistent sentiment–rating pairing after fine-tuning. This being the only category where all of them overperformed Deepseek.

- *Grammar Errors*: Interestingly, the non-fine-tuned models (Deepseek and Base Gemma) show substantially fewer grammatical mistakes per review (0.31 and 0.36, respectively) than the fine-tuned Gemma variants (e.g. a32,r64 QLoRA at up to 2.10 errors). This may be because the user-generated reviews used for fine-tuning contained their own typos, but it could also contribute positively to a more “human-like” feel in the generated text. The only exception to this is the Mistral model which despite being finetuned shows the least grammar errors at 0.23.

Genre Accuracy and Perplexity

- *Genre Classification (Precision/Recall/F1)*: Deepseek still achieves the strongest F1 0.66, but following that the multi-headed model obtains a respectable 0.61 F1. The most successful normal Gemma was our smallest one (a8 r16 finished), achieving F1 0.54. On the other hand our biggest Gemma (a512 r256) struggles with F1 = 0.33.
- *Perplexity*: The lower the better. Gemma a32 r64 dropout 0.05 shows the lowest perplexity (3.89), indicating high fluency. The models which are outside the norm are gemma a512 r256 (10.06) and Mistral (12.88).

Review examples

Here are example reviews generated by our finetuned models

Movie: *Thor*

Mistral: “*Thor* is a fun movie with great special effects, but nothing groundbreaking. Chris Hemsworth is perfect in the title role. The story line is typical superhero fare, but the characters and dialogue are interesting.”

Gemma a8r16: “I liked the movie overall, however the ending was a little anticlimactic.”

Gemma a64r128: “I bought this movie to support the actors I adore. I’ve watched the series and now I’m buying the DVDs to support them. I have watched it several times now and I’m waiting for the rest of the series.”

Movie: *Blue Planet*

Mistral: “A great introduction to the Earth and its place in the cosmos.”

Gemma a8r16: “I would say I love this movie but am a bit conflicted. It’s obviously well done and the picture quality is excellent, but the content leaves a bad taste in my mouth. The narration seems to be deliberately misleading and purposefully manipulative. The first part of the movie is just stunningly beautiful images of Earth. Then they start telling you about pollution, deforestation, etc. And then they imply that it’s our fault. It’s true that humans contribute to these things, but they leave out all the negative environmental events that have happened in the history of the planet. They conveniently ignore the supervolcanoes, massive meteor impacts, ice ages, floods, droughts, fires, tsunamis, plagues, famines, wars, pestilence, and so on. To add insult to injury, they end the movie with a

plea for donations, as if saying “give money to help stop these disasters.” That’s totally ridiculous. They should quit talking about saving the planet and work harder to save themselves.”

Gemma a64r128: “A movie that I would love to watch.”

As you can see the reviews generally align with the movies they are about, they can be long, short, they can say what the movie is about or not. Sometimes the models hallucinate some information, this happened in the end of the Gemma a8r16 Blue Planet review, where the model claims the movie asks for donations which does not happen.

All the reviews generated by the models can be found on our github in tests/new_datasets

Key Takeaways

- All fine-tuned Gemma variants substantially outperform the untuned Gemma baseline on rating prediction (MAE, RMSE, % within ± 1), genre-classification metrics and review quality, though they incur more grammar errors. However, aside from ratings, none of the fine-tuned models fully match Deepseek’s balance of semantic similarity and genre prediction accuracy.
- *Optimal Gemma configuration:* The **a32_r16** model achieves the best overall balance—lowest MAE (1.34), highest % within ± 1 (50.92%), strong F1 (0.55), and lowest perplexity (3.87). Regarding dropout rate, a lower value appears preferable: the **a32_r64** model with 0.05 dropout yields the lowest perplexity (3.89), maintains a high F1 (0.55) and its reviews score better, whereas the 0.2 dropout variant overall performs worse. We must also note that the **a8_r16** model, despite its smaller size, achieves overall great results, with its similarity to human reviews being the highest of all the finetuned models.
- The largest Gemma (**a512_r256**) under performs: it incurs the highest grammar error rate (2.21), the lowest genre F1 (0.33), and the highest perplexity (10.06), suggesting over-parameterization without sufficient regularization.
- There is almost no discernible difference between using 4 bit and 8 bit quantization, that can be seen from the test results. The only thing is maybe that the Gemma **a32_r64** QLoRA has double the grammatical errors, than its Gemma **a32_r64** counterpart.
- Mistral showcases strong linguistic quality yet poor rating prediction, suggesting a need for task-specific fine-tuning.
- The Multi-Head model delivers the strongest performance concerning rating and genre prediction MAE 0.96, RMSE 1.25, % within ± 1 64.40% and F1 0.61. This is despite the fact that we only use the main head and that it was trained on a limited dataset. We believe this is because to conserve resources we further limited the LLM components which were tuned, and this made a better performing model.

Model	MAE	RMSE	% within ± 1	Similarity	Sen. Mismatch
Deepseek	1.47	1.77	37.79%	0.42	46.93%
Base Gemma instruct	2.46	2.76	12.53%	0.284	84.93%
Gemma a8 r16	1.41	1.79	45.17%	0.372	38.34%
Gemma a16 r32	1.37	1.76	48.69%	0.357	35.90%
Gemma a32 r64	1.38	1.76	46.84%	0.360	39.43%
Gemma a32 r64 dropout 0.2	1.40	1.77	46.17%	0.353	40.71%
Gemma a32 r64 dropout 0.05	1.38	1.74	46.72%	0.363	43.37%
Gemma a32 r64 QLoRA	1.44	1.85	47.42%	0.361	35.05%
Gemma a32 r16	1.34	1.73	50.92%	0.363	39.33%
Gemma a32 r32	1.41	1.79	46.50%	0.362	42.99%
Gemma a64 r128	1.41	1.81	46.65%	0.347	41.02%
Gemma a512 r256	1.46	1.87	45.10%	0.360	42.80%
Multi-Head	0.96	1.25	64.40%	0.361	41.48%
Mistral 7B	1.90	2.34	29.73%	0.350	28.96%

Model	Grammar Err.	Precision	Recall	F1	Perplexity
Deepseek	0.31	0.58	0.78	0.66	–
Base Gemma instruct	0.36	0.25	0.40	0.30	19.53
Gemma a8 r16	0.91	0.55	0.52	0.54	4.01
Gemma a16 r32	1.88	0.51	0.49	0.50	4.19
Gemma a32 r64	0.97	0.51	0.50	0.50	3.96
Gemma a32 r64 dropout 0.2	1.69	0.50	0.47	0.49	4.62
Gemma a32 r64 dropout 0.05	1.16	0.57	0.54	0.55	3.89
Gemma a32 r64 QLoRA	2.10	0.50	0.50	0.50	3.95
Gemma a32 r16	1.48	0.58	0.53	0.55	3.87
Gemma a32 r32	1.95	0.55	0.54	0.55	4.60
Gemma a64 r128	1.30	0.46	0.45	0.45	3.96
Gemma a512 r256	2.21	0.34	0.31	0.33	10.06
Multi-Head	0.51	0.64	0.58	0.61	5.82
Mistral 7B	0.23	0.45	0.37	0.40	12.88

Table 1.3: Results of all model testing

1.13 Conclusion

With the numerous hurdles on the way, overall we think that we’ve succesfully completed the assignment, we have a working model. But with everything in life our solution could also be further improved. We were unable to surpass the baseline model. But considering that we are doing this for the first time, and we were able to come close to the baseline results using a model which is less than 1% in size. Also the resulting model can easily be ran on decent consumer grade hardware, not requiring multiple servers, or expensive components.

Further improvements:

- Do more experiments with the results from the multi head model – Considering that this was the best performing model our next experiment would have been to train a model without the heads but only train chosen components.

- Replace reviews with higher quality ones – This was not done due to time constraints, but considering that the assignment was to create a language model and evaluate the results, we think that this is currently not an issue. For real world use this task should have a higher priority, but we think that the results with this dataset would translate well to the better one.
- Try larger variants of the model – During evaluation hallucinations were common, this may be caused by the relatively small model used. In the future larger models from the Gemma family should also be fine-tuned, to find a sweet spot, where hallucinations occur a lot less.

Overall we really enjoyed working on this project, it had its ups and downs but it was really interesting to practically try out the things we've learned on the lectures. All of the finetuned models are available at https://dns.h10s.eu/KNN/KNN_models.html.

1.14 Links

- GitHub: https://github.com/Martin-Toma/KNN_project
- Dataset: <https://akirabox.com/9QWmpZ7XzEB6/file>
- Paper: <https://aclanthology.org/2024.lrec-main.206/>
- Report: <https://www.overleaf.com/read/kpckzfrfncst#a4efae>
- Poster: <https://www.overleaf.com/read/wtmjmjbjbkcpz#40c6e8>
- Final models: https://dns.h10s.eu/KNN/KNN_models.html